

SQA



Đảm bảo chất lượng phần mềm

Software Quality Assurance



3. Đảm bảo chất lượng phần mềm trong dự án

Nguyễn Anh Hào

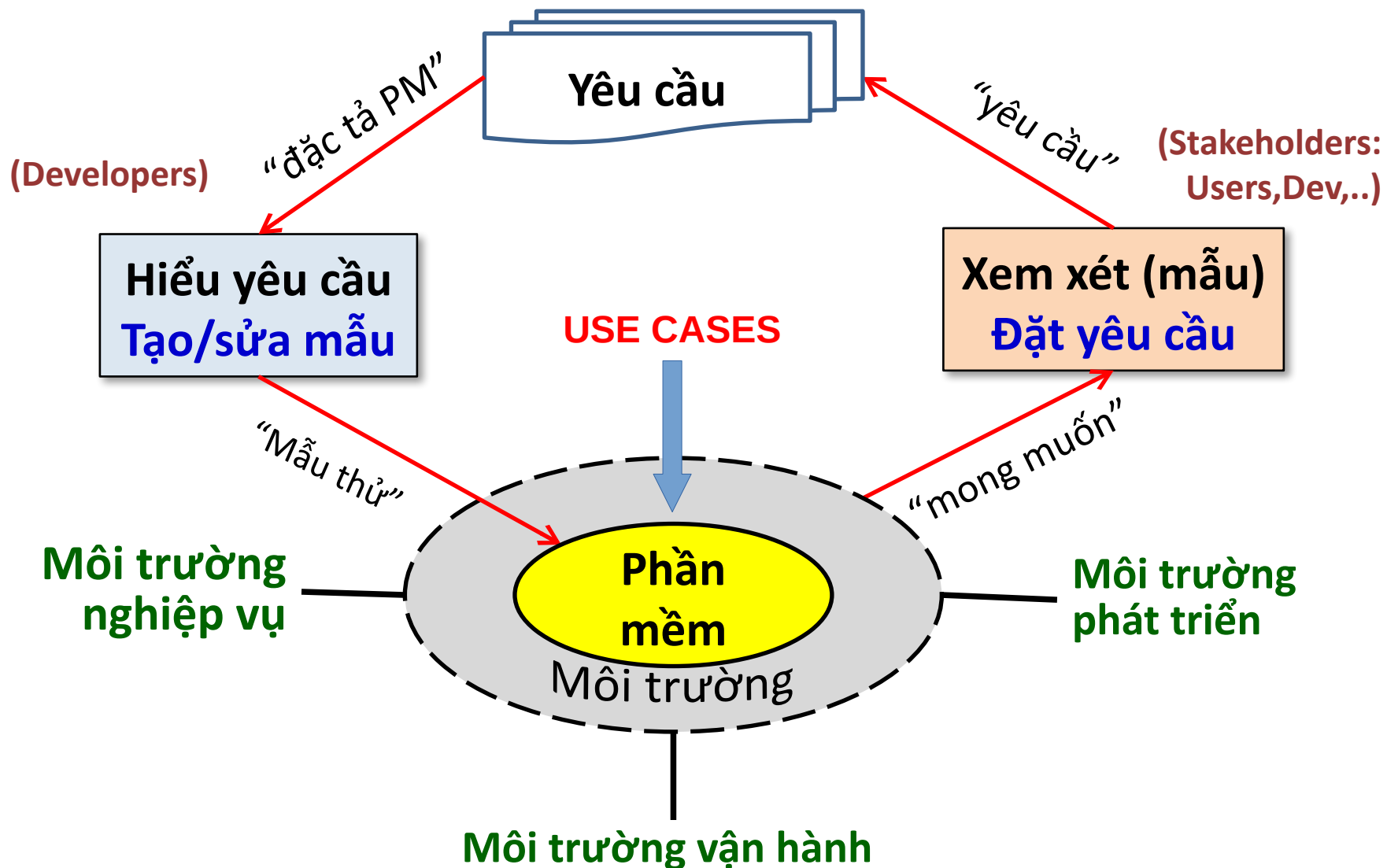
Khoa CNTT2, Học viện Công Nghệ BCVT Tp.HCM

✉: nahao@ptithcm.edu.vn

☎: 0913609730

Tài liệu môn học – 2025

a) Xác định yêu cầu cho phần mềm

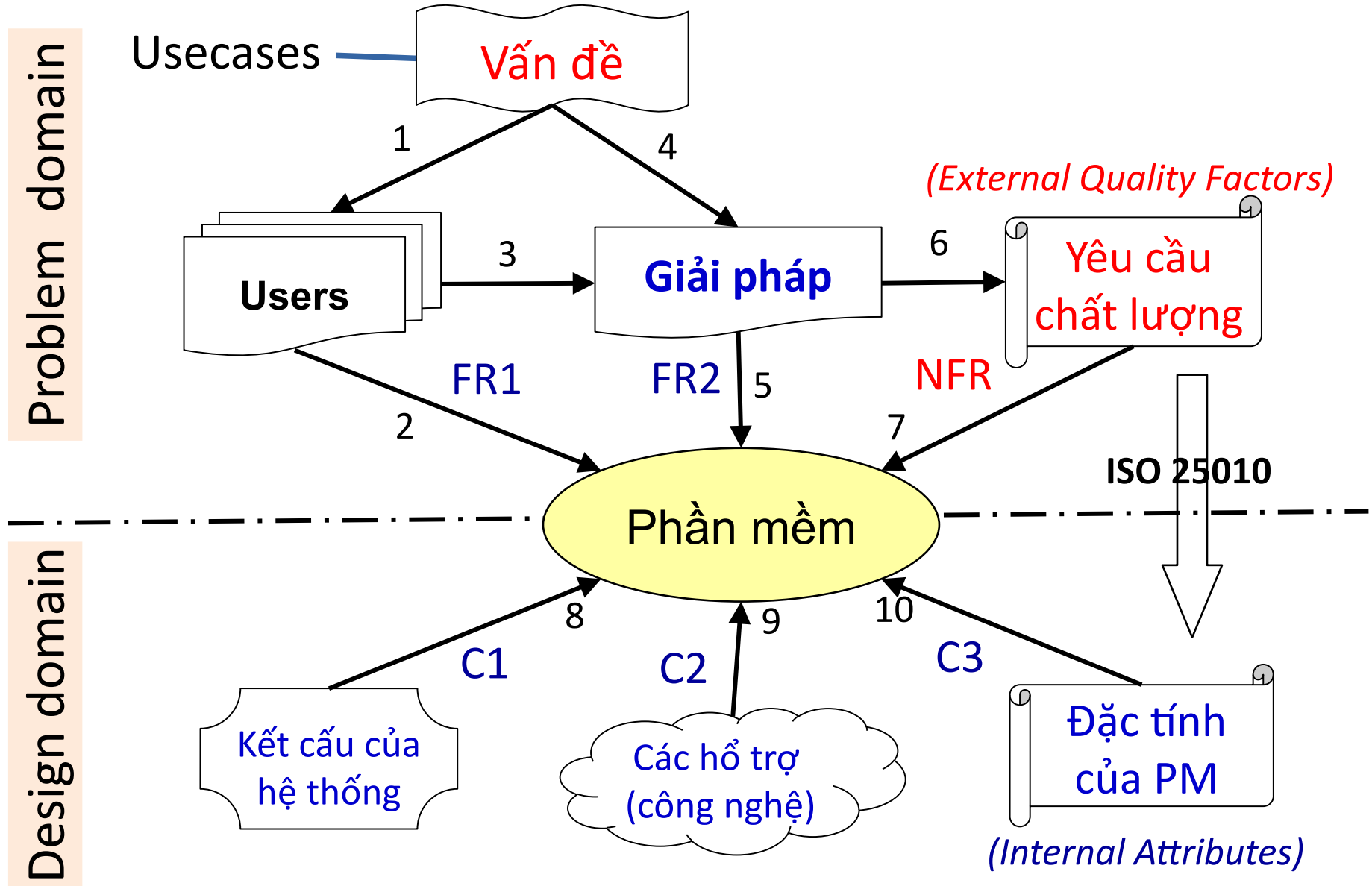


Mô hình này dùng để đặt ra yêu cầu & kiểm chứng yêu cầu (validation)

- **Môi trường nghiệp vụ** là môi trường làm việc của người sử dụng, có những vấn đề “**bất biến**” cần giải quyết (trong usecase) là nguồn gốc gây ra yêu cầu làm PM.
 - Cần xác định đúng **vấn đề** để có yêu cầu đúng cho PM.
 - Cần biết đầy đủ các **ràng buộc** để xây dựng giải pháp tốt
- **Môi trường vận hành** là môi trường thực thi PM: quy trình vận hành, người dùng, lớp nền, hệ thống khác,...
 - PM phải **tương thích** và **cải tiến** trong môi trường này
- Người phát triển xây dựng PM bằng cách tìm kiếm và sử dụng các hỗ trợ từ công nghệ, phương pháp, cộng đồng,... trong **môi trường phát triển** của PM.
 - Cần cộng đồng các chuyên gia (**devs+users+experts**) và các hỗ trợ (công cụ, giải pháp) có giá trị lâu dài

b) Tìm giải pháp cho phần mềm

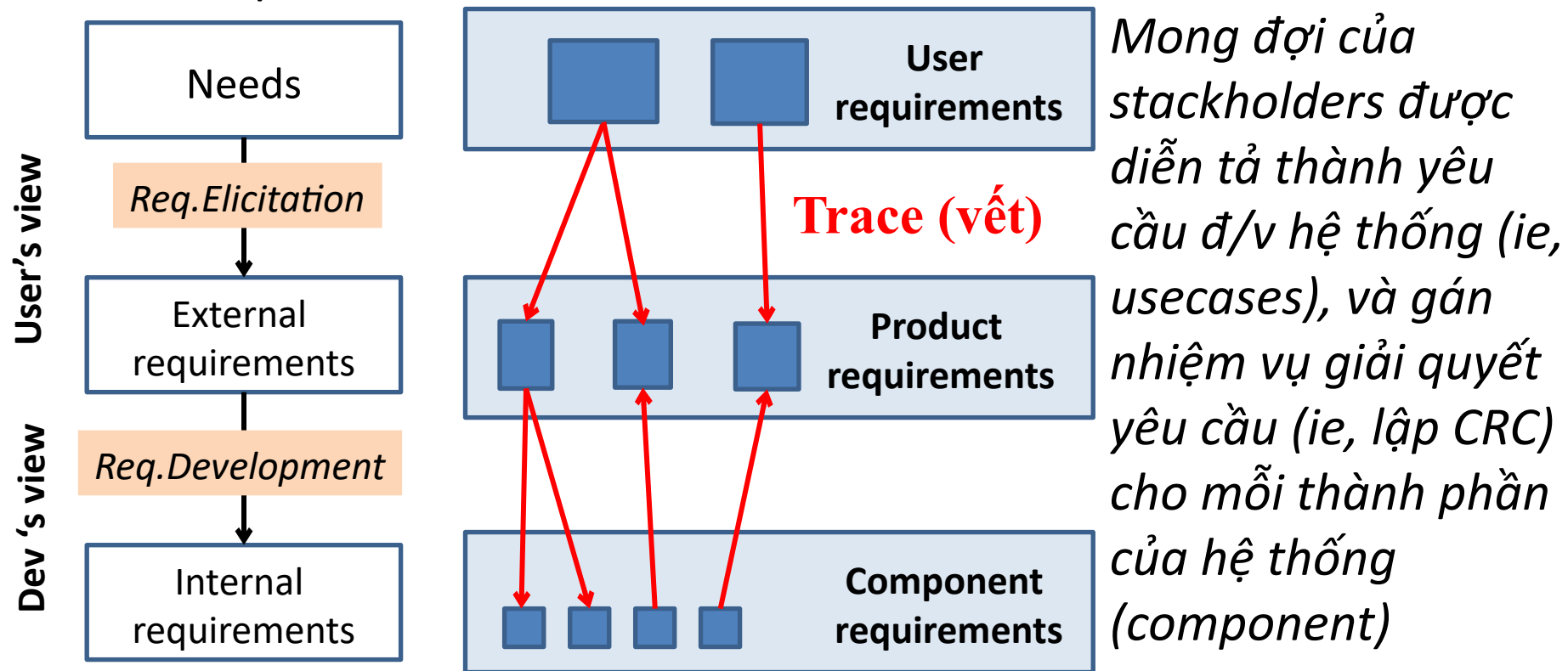
5



c) Chi tiết hóa yêu cầu thành đặc tả

Cụ thể hóa yêu cầu thành **đặc tả chi tiết ở các mức** (mức ý niệm, mức thiết kế, mức lập trình,..) nêu ra trong các tài liệu (để rà soát và cài đặt).

SW spec.



Yêu cầu giữa các mức có liên kết nhau (trace)

(Vết và dò vết –Traceability- được trình bày trong phần riêng)

d) Kiểm chứng (rà soát) các đặc tả

- Là hành động chứng minh rằng các đặc tả cho PM là đúng. Hành động này phân tích mối quan hệ giữa PM với các môi trường sống của nó, để khẳng định rằng các đặc tả này là **cần thiết, đúng và đầy đủ**.
- Phương pháp:
 - *Validation*: CMR sản phẩm sẽ thỏa mãn mong muốn
 - *Verification*: CMR hành động làm sản phẩm là đúng.
- Kỹ thuật:
 - Peer review
 - Inspection

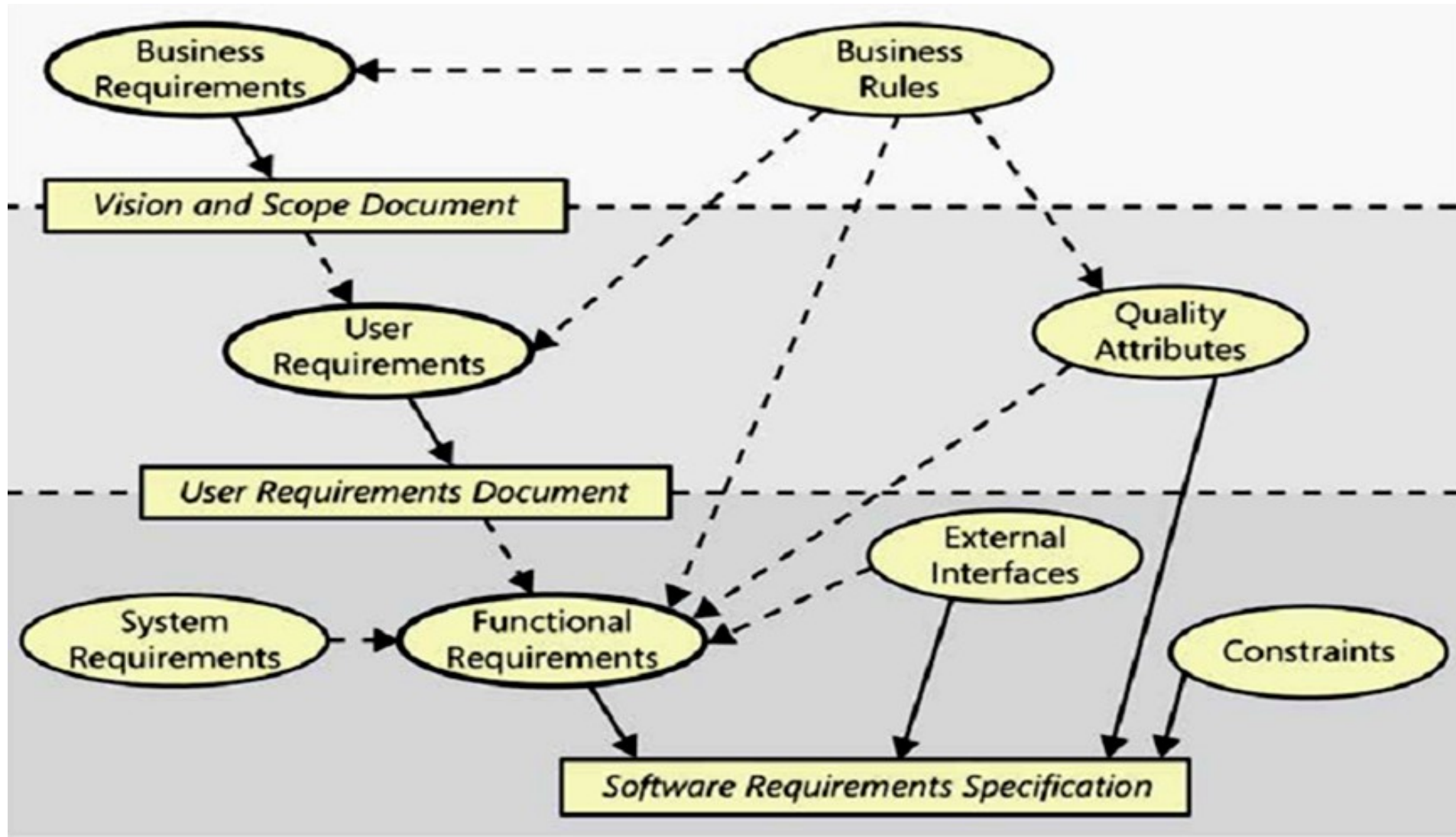
e) Kiểm thử phần mềm

- Là hành động chứng minh rằng sản phẩm PM đã được thực hiện đúng theo các đặc tả, và thỏa mãn mọi mong đợi trong môi trường sống của nó.
- Phương pháp kiểm thử:
 - Black-box testing: đối chiếu kết quả với kỳ vọng
 - White-box testing: xem xét kết cấu bên trong
 - Gray-box testing: dùng white-box để làm black-box test
- Kỹ thuật:
 - Black-box: equivalence partitioning, boundary value analysis, decision table, Cause–Effect Graphing (CEG),...
 - White-box: path testing, condition testing
 - Gieo lỗi, để đo khả năng kiểm thử

Dự án: Xác định yêu cầu làm PM

Nhận định rõ về bối cảnh phát sinh vấn đề (use case), yêu cầu và giải pháp cho PM

- **Xem xét** nhu cầu sử dụng PM từ **môi trường nghiệp vụ**, khả năng triển khai áp dụng PM từ **môi trường vận hành**, cách thức xây dựng PM trong **môi trường phát triển** từ khía cạnh học thuật, mô hình phát triển, công nghệ và chuẩn ... để đưa ra nhận định về vấn đề và giải pháp.
- So sánh các chức năng, giải pháp của PM dự kiến làm với các PM tương tự đã có trên thế giới để định hướng phát triển, và cập nhật nâng cấp



Dự án: Thiết kế PM

Chỉ ra giải pháp của PM cho các vấn đề đã biết, và đặc tả chi tiết để xây dựng PM, kiểm thử và triển khai một phiên bản PM:

- Đặc tả các **chức năng và kết cấu** của PM (các usecase, class, layers, package/subsystem,...)
- Đặc tả **chi tiết để lập trình** (mô đun, dữ liệu, giải thuật, giao tiếp, công nghệ / chuẩn,...)
- Đặc tả **chi tiết kiểm thử**: test cases, test plans
- Đặc tả **cách vận hành** của PM (mô hình vận hành, các vai trò của users, flat-forms, giao tiếp với các hệ thống khác,...)

Tất cả các hành động đưa ra đặc tả đều phải được chứng minh là đúng qua rà soát (thỏa mãn cả FR lẫn NFR)

❶ Cần phát hiện những yêu cầu “chắc chắn đúng” cho PM **càng sớm càng tốt** để làm cơ sở phát triển các yêu cầu khác.

1. Yêu cầu đúng là yêu cầu dẫn xuất từ bản chất của vấn đề, không hoặc ít bị thay đổi (**invariance**, bất biến).
2. Tạo điều kiện cho users (và các stack-holders) tiếp cận sớm với PM/mẫu thử (**user involvement**) để họ tư vấn/gợi ý/đánh giá:
 - *Nhận định về vấn đề bị sai, hoặc còn thiếu*
 - *Đánh giá tầm quan trọng của vấn đề không đúng*
 - *Giải pháp của phần mềm chưa tốt*
 - ...

- ② Xem xét toàn diện các khía cạnh phát triển, chuyển giao, vận hành, tiến hóa của PM để đưa ra đặc tả, bằng cách **phối hợp nhiều chuyên gia**.
- Có phân chia trách nhiệm rõ ràng, có phản biện
 - Có phương tiện hỗ trợ hợp tác: forums, Figma/Penpot
 - Các website stackoverflow.com, [expertExchange.com](https://expertexchange.com), [codeProject.com](https://codeproject.com), hoặc [gitHub.com](https://github.com) đều cung cấp các tư vấn chuyên sâu cho giải pháp.
 - Sử dụng Chat-bot để kiểm thử.

Ý kiến từ người sử dụng được ưu tiên, vì nó thường diễn tả trực tiếp yêu cầu từ tổ chức (nghiệp vụ, môi trường vận hành, hiệu quả dùng tài nguyên,...), tuy nhiên người sử dụng không giúp giải quyết được vấn đề kỹ thuật (công nghệ, an toàn, cải tiến,...).

- ③ Làm cho việc hiểu biết về PM dễ dàng hơn bằng tài liệu và mẫu thử.
 - Làm phim minh họa (user story)
 - Dùng CASE tools để thiết kế
 - Source code viết theo chuẩn, có chú thích đầy đủ (kể cả các lần cập nhật)
- ④ Tài liệu phát triển PM được mô tả một cách có hệ thống; sự “phiên dịch” yêu cầu ban đầu thành đặc tả chi tiết cho giải pháp (sản phẩm) phải đúng và đủ.
 - **Đúng**: Mọi ý trong các tài liệu đều nhất quán (**consistency**).
 - **Đủ**: Mỗi yêu cầu được cụ thể hóa thành giải pháp chi tiết trong mức thấp hơn, nhưng **không dư thừa**.